GraphQL-Shield: Dual-Layer Enforcement Against GraphQL BOLA with Cross-Layer Coverage Evidence

[Author 1]
[Department, Institution]
[City, Country]
[email]

[Author 2]
[Department, Institution]
[City, Country]
[email]

[Author 3]
[Department, Institution]
[City, Country]
[email]

***Abstract*—Broken Object Level Authorization (BOLA) is the top risk in the OWASP API Security Top 10, and GraphQL makes it harder to stop: one request can reach an object by identifier, through nested lookups, fragments, variables and batches, or through relationship fields that never name the object. We present GraphQL-Shield, which enforces object ownership at two independent points: a schema-aware layer that checks every identifier it can resolve before execution, and an execution-time layer that checks every object resolvers return and every mutation target the engine coerces. The second layer records each object the first never evaluated, so the gateway measures its own pre-execution coverage. We test this evidence by fault injection: 17 realistic bugs planted in the first layer, each run on 5 seeds of 1,000 composed requests. The evidence revealed all 17 and raised no alarm on the unmodified layer. A one-line rule over the evidence predicted the affected requests with mean F1 0.966; logistic regression and XGBoost both named the broken construct first for all 14 faults our features can express, and XGBoost was no better than the simple models. Turned into configuration repairs and checked by replaying recorded requests offline, the same evidence restored all 10 configuration mistakes we planted in two applications (30 of 30 runs over three seeds), proposed no repair for the correct configurations, and refused all 33 wrong decoy repairs. We state four security properties and check them with property-based testing, close four leak channels the tests exposed (an existence oracle, objects behind interfaces and unions, aggregate fields, and a refused folder deletion), measure no timing difference between foreign and missing objects, and run the same layers on Strawberry, plain graphql-core, Graphene and Ariadne. On the same schema and 13 attacks, graphql-shield also leaks nothing but loses an owner's own data to GraphQL null propagation; GraphQL-Shield loses none, but a type left out of its configuration leaks up to 9 of 13 attacks.**

***Index Terms*—GraphQL, API security, broken object level authorization, access control, defense in depth, fault injection, property-based testing**

I. Introduction

An API has a BOLA flaw when it returns or changes an object without checking that the caller may access it. OWASP ranks it first among API risks (API1:2023) [1]. It is hard to detect because a BOLA request is ordinary authenticated traffic: the only difference is whose object it names.

GraphQL widens this attack surface. A REST endpoint usually takes one identifier in its path; a GraphQL request can name many objects at any depth and can reach objects without naming them. A caller who owns a folder can list its `children`, including a document owned by someone else, and no identifier for that document appears in the query. The same identifier can also arrive as a literal, a variable, a variable's default value, inside a named or inline fragment, under an alias, or in one operation of a batch.

Two families of defence exist, and each misses something. Checks that analyse the query before execution see only the identifiers written in it, so relationship traversal is invisible to them. Checks written into each resolver see every object but must be written and maintained field by field. GraphQL-Shield combines both and uses the second to audit the first. Our contributions are:

1) **Dual-layer enforcement with cross-layer evidence.** A pre-execution layer and an execution-time layer enforce ownership independently, configured from the schema and a small ownership map. The second records every object the first never evaluated, which turns defence in depth into a coverage measurement.
2) **A fault-injection evaluation of that evidence.** We plant 17 bugs in the pre-execution layer and measure detection, prediction and localisation. The evidence detects every fault; simple models over it are as good as gradient boosting.
3) **Self-repairing configuration.** The same evidence names the configuration entry a gap comes from and proposes a repair, which is verified by replaying recorded requests through the pre-execution layer offline and applied only by an administrator.
4) **Stated properties and closed leak channels.** We define safety, non-interference, write safety and availability, check them on generated data and requests, show they fail with protection off, and fix four channels that returned or revealed foreign data in an earlier version.
5) **Portability and comparison.** The same layers run on four Python GraphQL frameworks and a second, unrelated application. We compare with graphql-shield on the same schema, including configuration mistakes, and

TABLE I
FORMS OF BOLA IN GRAPHQL

Form	Example	Target ID in query?
Direct	`resource(id:102)`	Yes
Nested	`user(id:101){ resource(id:102)}`	Yes, below the root
Syntactic variant	variable, default, alias, fragment, directive, batch	Only after coercion
Relationship	`resource(id:201){ children{...}}`	No
Write	`updateResource(id:102,...)`	Yes, as an argument

report latency with confidence intervals.

II. BACKGROUND

GraphQL execution. A server parses a request, validates it against a typed schema, and calls a resolver for each selected field, top down [2]. Arguments are coerced first: variables are substituted, missing variables take their declared defaults, and fragments, aliases and `@include`/`@skip` directives are expanded. A defence that reads the query text sees the request before coercion; a defence inside the engine sees the values resolvers receive and return.

Forms of BOLA in GraphQL. Table I lists the five forms used throughout the paper.

Global identifiers and type confusion. Many GraphQL APIs use Relay-style opaque identifiers, typically base64 of `Type:id`. A resolver that decodes one without checking its type can be handed an identifier of one type where another is expected; Saleor's CVE-2022-39275 is an instance [3].

Abstract types. A field typed as an interface or union (`node(id:)`, a search result) returns objects of several concrete types. A check that reads only the declared return type sees no owned type there.

Null propagation. When a non-null field raises an error, GraphQL nulls its nearest nullable ancestor [4]. In a list typed `[T!]!`, denying one item cannot null that item alone, so the error nulls the parent object, and the caller's own objects disappear with it.

III. RELATED WORK

We found no prior work that enforces BOLA in GraphQL at two layers and uses one layer to measure the other's coverage.

BOLA detection by testing. GraphQLer builds a dependency graph from introspection, chains queries and mutations, and replays requests as other users [5]. AuthProbe tests REST APIs from an OpenAPI specification with multiple identities [6], and a Colored Petri Net approach derives attack models from OpenAPI specifications [7]. These are test-time tools, two of them for REST. GraphQL-Shield is a runtime defence such tools can evaluate.

BOLA enforcement and analysis. BolaZ combines CodeQL taint tracking with runtime authorization checks in Spring Boot services [8]; it shares our view that static analysis needs a runtime backstop but targets Java REST services and does not use one layer to measure the other. Lambers et al. define taint analysis for graph APIs aimed at broken access control [9], a static complement to our runtime checks.

GraphQL authorization and query analysis. graphql-shield attaches permission rules to types and fields as resolver middleware [10]; it is the per-field approach we compare against. Formal semantics [2], static cost analysis [11] and empirical schema studies [12] underpin depth and cost limits, which bound how expensive a query is, not whose objects it reaches.

Foundations. Layer 2 is a reference monitor [13] that mediates every returned object, following complete mediation and fail-safe defaults [14]. Our non-interference property follows Goguen and Meseguer [15]. Comparing two layers' decisions is a form of differential testing [16], planting bugs to judge a detector follows mutation testing [17], and we check properties as in QuickCheck [18] using Hypothesis [19]. Turning automatically produced records into training labels resembles weak supervision [20]. A taxonomy of over 100 bug-bounty BOLA disclosures includes GraphQL global-identifier leakage [21].

IV. THREAT MODEL AND PROPERTIES

The attacker is an authenticated user who wants to read or change objects owned by someone else. They hold a valid token for their own account, can send any syntactically valid request, know or can guess other users' identifiers, and can read the schema. Authentication is sound and the owner lookup is trusted: ownership comes from the data store, never from client input or from the object a resolver returns. Resolvers may contain no authorization checks. An owned object is accessible to its owner and to administrators, and objects reached through relationships are checked individually: owning a parent grants nothing about its children. Out of scope are authentication flaws, function-level authorization, injection, denial of service beyond the depth, alias, token and batch limits we apply, timing channels other than the foreign-versus-missing comparison we measure, and policies richer than owner-or-administrator.

For a non-administrator caller u and any request q we require:

- P1 Safety. The response to q contains no object u does not own.
- P2 Non-interference. Changing only objects u does not own (their content, existence, or children placed under u's folders) does not change the response to q.
- P3 Write safety. q changes the owner or content of no object u does not own, and deletes none.
- P4 Availability. If q names only objects u owns, they are returned.

P2 is stronger than P1: it rules out answers that reveal foreign data indirectly, such as an error that differs for a foreign object and a missing one, or a count that includes foreign children.

V. DESIGN

Fig. 1 shows the architecture: two independent checks around the GraphQL engine, and a record of where they disagree.

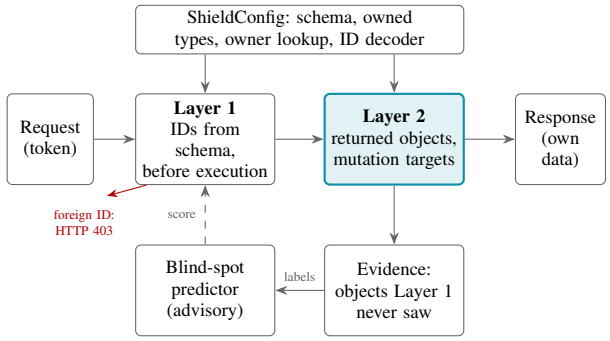


Fig. 1. GraphQL-Shield. Layer 1 rejects requests naming foreign objects (HTTP 403); Layer 2 filters returned objects, blocks foreign writes, and records every object Layer 1 never evaluated.

A. Per-application configuration

A `ShieldConfig` describes the protected application: its schema; which object types are owned, each mapped to an ownership key; an owner-lookup function from (key, identifier) to owner, optionally with a batched form; the arguments that carry identifiers and a decoder for them (for example, Relay global identifiers); targets for mutations whose return type does not reveal what they change; and aggregate fields, such as a child count, that summarise owned objects. Everything else is derived from the schema. The demo application needs two owned types, one mutation entry (`deleteResource` returns `Boolean`), one aggregate entry and its owner lookup.

At startup a lint pass compares the configuration with the schema. It reports object types not configured as owned that have an owner-like field (`ownerId`, `author`, `createdBy`, ...) or a field of an owned type, and scalar fields on owned types named like counts or totals (`childrenCount`, `hasChildren`) that are not declared as aggregates. Independently of naming, it also reports every root query that fetches a single object of a type by identifier when that type is neither owned nor declared public, since a per-user lookup is the usual sign of an owned type. Removing any one of the five owned types across our two applications produces a finding for that type, and the complete configurations produce none. The lint remains a heuristic: a type with an unusually named owner field that is reachable only through relationships is not reported.

B. Layer 1: schema-aware pre-execution gate

Layer 1 parses the request before the engine runs and walks it against the schema. For every field with an identifier argument it takes the object type from the schema—the owned type the field returns, the type embedded in a global identifier when the field returns an interface or union, or a root mutation's configured target—rather than from the field's name. It resolves values as the engine will: literals, variables, declared defaults, lists, named and inline fragments, aliases, and each operation of a batch. A global identifier whose embedded type differs from the expected type is checked as both. One denial rejects the whole request with HTTP 403, so a write never starts.

C. Layer 2: execution-time guard

Layer 2 runs inside the engine as graphql-core middleware or a Strawberry extension. Before a root mutation's resolver runs, it checks the target argument the engine coerced; a denial raises an error and the resolver never executes. After each resolver returns, it decides whether the value is an owned object from the field's GraphQL type, not the value's class, so it works with typed objects, dictionaries or ORM rows. For an interface or union it resolves the concrete type the way the engine does (`__typename`, `resolve_type` or `is_type_of`). A denied single object becomes `null`; a denied list item is removed before non-null types are enforced, so the caller keeps their own items. A list is checked with one batched owner lookup and its decisions are written in one audit transaction. A declared aggregate field is answered by Layer 2 itself as the number of related objects the caller may see. A mutation with no derivable or configured target is denied unless explicitly listed as unguarded.

D. Uniform denials

Every denial a non-administrator sees carries the same reason, `ACCESS_DENIED`, whether the object belongs to someone else or does not exist; the exact reason and risk score are kept only in the audit log. An earlier version returned `OWNERSHIP_MISMATCH` or `OBJECT_NOT_FOUND`, and its request explainer returned the owner's identifier, which together let any user enumerate which objects exist and who owns them. Denials also take the same time: when a lookup by identifier finds nothing, Layer 2 still performs the owner lookup, policy decision and audit write it performs for a foreign object.

E. Cross-layer evidence and the advisory predictor

Layer 1 marks every (type, identifier) it evaluated. Layer 2 writes an audit record only for objects Layer 1 never marked, so each such record is direct evidence of a Layer 1 coverage gap, and no decision is counted twice. When Layer 2 also denies such an object it records a blind-spot event: an attack only Layer 2 stopped. The audit log is hash-chained, so a deleted or edited record is detectable.

The evidence labels requests without manual effort. For each request Layer 1 allows we compute 17 structural features available before execution: depth, field count, relationship fields and chain depth, identifier arguments and how many Layer 1 resolved, nested lookups, variables and defaults, fragments, aliases, directives, batch size, mutation flag and caller role. The features never include ownership data or outcomes. An XGBoost classifier [22] (120 trees, depth 4) predicts whether a request will reach objects Layer 1 cannot see, and SHAP contributions [23] rank the features behind each prediction. At runtime the score is advisory: it is recorded with later blind-spot events and never changes a decision.

F. Self-repairing configuration

Most coverage gaps come from the configuration, not the code, and the evidence says which entry. For each object

Layer 1 missed, Layer 2 records the field and the coerced arguments that reached it, and it records every mutation it refused for lack of a target. Recent requests are sampled in memory. A repair module assigns each gap a cause. An argument whose value equals the missed object's identifier but is not listed as an identifier argument yields a repair that lists it. Refused calls to a mutation yield a target entry: the argument whose decoded value the owner lookup resolves, and the owned type, preferring the type embedded in a global identifier and the one the mutation's name mentions. A type the startup lint flags yields an owned-type entry. Relationship traversal is labelled expected. A listed argument that Layer 1 still missed is reported as a code gap, with the query constructs the requests share.

A repair is verified before it is shown. The recorded requests are replayed through Layer 1 with and without it; nothing executes. It must close gaps or protect objects, block no request that Layer 2 had let through unchanged, have at least 90% of the identifiers it newly reads recognised by the owner lookup, and not give a type an ownership key another owned type already uses. An administrator applies a repair, which is verified again, stored, re-applied at startup and revertible. Nothing is applied automatically.

VI. Implementation

GraphQL-Shield is written in Python on FastAPI and graphql-core 3.3, with SQLite (write-ahead logging) for ownership data and audit records. Layer 1 is a dependency on the protected route; Layer 2 is `OwnershipMiddleware(config)` for any graphql-core server or a Strawberry extension. Every request is limited to depth 10, 15 aliases, 2,000 tokens and 5 batched operations. Database connections are pooled, parsed queries are cached, and the advisory score is cached per feature vector. Sign-in is rate-limited per account and per client address, tokens can be revoked, and every request re-reads the caller's role from the database, which the threat model's assumption of sound authentication relies on. The test suite has 427 tests.

We evaluate on two applications. The *demo* application (Strawberry, code-first) has users, documents and folders with integer identifiers. The *clinic* application shares no code with it: a schema-first SDL on plain graphql-core, dictionary resolvers, Relay global identifiers, and patients, records and prescriptions with independent owners. In both, resolvers contain no authorization checks, and each exposes an unprotected endpoint used only as a baseline. To test framework independence we also protect small Graphene (Relay `Node`, lists, mutations) and Ariadne (schema-first) applications with the same middleware and configuration type.

VII. Evaluation

We address six questions. **RQ1:** does each layer stop BOLA alone, and do both together? **RQ2:** does the cross-layer evidence reveal faults in Layer 1, and what is needed to use it? **RQ3:** do the stated properties hold? **RQ4:** how does the approach compare with per-field rules, including under configuration mistakes? **RQ5:** what does enforcement cost? **RQ6:** can the evidence repair a mistaken configuration? All experiments run in-process on temporary seeded databases on one Windows laptop with Python 3.13. Deterministic experiments report counts; randomised ones report the mean and spread over seeds.

TABLE II
Layer ablation: the layer that stopped each attack

Attack	Both	L1 only	L2 only	Neither
Direct	L1	L1	L2	leaked
Nested lookup	L1	L1	L2	leaked
Inline fragment	L1	L1	L2	leaked
Relationship	L2	**leaked**	L2	leaked
Batch	L1	L1	L2	leaked
Update mutation	L1	L1	L2	leaked

A. RQ1: Layer ablation

Table II runs six attacks by one user against another user's objects under each layer combination. Layer 1 alone leaks relationship traversal, because no identifier of the child appears in the query; Layer 2 alone stops every attack but catches writes only at execution time. The clinic application, protected through its configuration alone (three owned types, two mutation entries), passes all 14 of its security tests, including type-confused global identifiers, one- and two-level relationship filtering, and Layer 2 acting alone. The Graphene and Ariadne applications pass their tests for foreign single objects, filtered lists, foreign mutations that never run, and Layer 1 reading their schemas.

B. RQ2: Cross-layer evidence as a fault detector

One discovery. We generated 4,000 varied requests from four callers and labelled them from Layer 2's evidence. Layer 1 ignored variables' declared default values, so `query($id: Int = 102)` sent without variables reached object 102 unchecked. All 585 such requests were coverage gaps and 263 were attacks stopped only by Layer 2. The hand-written rule for Layer 1's documented blind spot, "flag requests with a relationship field", reached F1 0.639 and caught 22.8% of these attacks; the model reached F1 1.000. After the fix, attacks reaching only Layer 2 fell from 401 to 138, the remaining gaps were relationship traversal, and the rule matched the model (F1 0.996 for both). Over five further seeds of 2,000 requests the model and the rule tied at F1 0.992 ± 0.006.

Fault injection. One discovery is an anecdote, so we measured the evidence systematically. We wrote 17 faults a developer could plausibly introduce into Layer 1 (Table III), each patched into its source at load time, and ran each fault and an unmodified control on five seeds of 1,000 requests with Layer 2 on. Requests are composed from independent constructs—identifier source (literal, variable, default, list), wrapper (alias, directive, named or inline fragment, nested fragment), location (root, nested lookup, relationship), shape (single operation or batch) and operation (query or

TABLE III
FAULT INJECTION: 17 LAYER 1 FAULTS, 5 SEEDS OF 1,000 REQUESTS EACH

	Result	Over
Faults detected by evidence alone	17 / 17	5 / 5 seeds each
False alarms on unmodified Layer 1	0	5 seeds
Prediction, mean F1 over faults		
Relationship rule	0.661	
Evidence rule	0.966	
Decision tree (structural)	0.957	
Logistic regression (structural)	0.968	
XGBoost (structural)	0.967	
XGBoost (all features)	0.978	
Localisation, top-1 (14 nameable faults)		
Lift statistic	0.929	
Decision tree	0.957	top-3: 1.000
Logistic regression	1.000	
XGBoost	1.000	

mutation)—so a test request is not a copy of a training one. We ask three things of each fault. *Detection:* does the evidence contain gaps the known relationship blind spot does not explain? *Prediction:* on a held-out 30%, which requests will have gaps? *Localisation:* does a method's top-ranked feature, among gaps the relationship rule does not explain, name the broken construct?

The faults cover ignored variables, ignored defaults (everywhere or only in mutations), skipped named, nested or untyped inline fragments, skipped aliased or directive-bearing fields, nested lookups skipped or only the first visited, fragments handled only at the root, only the first operation, root field or batch operation checked, mutation targets skipped, and the user type skipped. Every fault left unexplained gaps on every seed, and the unmodified layer left none. The *evidence rule*—flag a request with a relationship field or an identifier argument Layer 1 did not resolve—reached a mean F1 of 0.966, against 0.661 for the relationship rule. It fails only where the fault makes Layer 1 believe it resolved an identifier it skipped (`first_batch_operation_only`, F1 0.733), which the learned models handle. Learned models did not beat simple ones: XGBoost on structural features (0.967) tied logistic regression (0.968), and adding Layer 1's resolution count raised XGBoost to 0.978. For localisation, logistic regression and XGBoost ranked the broken construct first for all 14 faults a feature can name. The other three faults (only the first operation, only the first root field, the user type skipped) have no feature that names them; they are detected but not localised, giving a top-1 rate of 0.824 over all 17.

The lesson is that the contribution is the evidence, not the model. A gateway with two layers can detect a broken first layer from its own logs, and a linear model or a one-line rule extracts most of the value.

TABLE IV
MEDIAN TIME OF A FOREIGN MINUS A MISSING OBJECT (MS), 1,500 PAIRS EACH

Setting	Gap	95% CI	p
Read, both layers	$+0.021$	$[-0.024, 0.060]$	0.98
Update, both layers	$+0.011$	$[-0.049, 0.065]$	0.87
Read, Layer 2 only	-0.005	$[-0.134, 0.122]$	0.68
Update, Layer 2 only	$+0.013$	$[-0.171, 0.223]$	0.70
Read, Layer 2 only, before fix	$+0.352$	$[0.193, 0.554]$	2×10^{-8}

C. RQ3: Properties and leak channels

We check P1–P4 with Hypothesis [19]. Each example draws a world—owners for three folders and six documents among three users, and a parent for each document—a caller, and a request composed as in Section VII-B; each property runs 120 examples. P2 compares the response in two worlds that agree on the caller's objects and differ arbitrarily in everyone else's. All four properties hold. With both layers off, each property fails, so the tests can detect a violation.

Writing P2 exposed four channels through which an earlier version revealed foreign data, all now closed. *Existence oracle:* Layer 1, Layer 2 and the explainer reported different reasons for a foreign object and a missing one, and the explainer returned the owner's identifier; responses are now identical apart from the requested identifier. *Abstract types:* Layer 2 read only the declared return type, so `node(id:2)` or a union search returned a foreign object's content; it now resolves the concrete type. *Aggregates:* `childrenCount` counted foreign children, and was protected only by a hand-written check in its resolver; Layer 2 now answers declared aggregates from the visible objects. *Refused deletion:* deleting a folder that held a hidden foreign child failed on the foreign-key constraint, revealing the child; a deleted folder's children now move to the top level, keeping their owner and content, so the response no longer depends on them.

Timing. Identical responses can still differ in time. For each of four settings we sent 1,500 interleaved pairs of requests for a foreign object and a missing one in random order, and compared the medians with a bootstrap 95% interval and a Mann–Whitney test (Table IV). With Layer 2 alone, a foreign read took 0.352 ms longer than a missing one (CI [0.193, 0.554], $p = 2 \times 10^{-8}$), because a lookup that found nothing skipped the check. After Layer 2 began checking missing lookups too, no setting showed a difference: every interval contains zero and every $p \geq 0.68$. The measurement is in-process; its resolution, about 0.1 ms, bounds what it can rule out.

D. RQ4: Comparison with graphql-shield

We rebuilt the demo schema, seed data and unchecked resolvers in Node.js and protected them with graphql-shield 7.6.5 in its documented style: argument rules on lookups, type-wide rules on owned types, and rules on each mutation. Both systems ran the same 13 attack and 5 benign

TABLE V
COMPARISON WITH GRAPHQL-SHIELD

	GraphQL-Shield	graphql-shield
Attacks leaked	0 / 13	0 / 13
Legitimate requests failed	0 / 5	1 / 5
Configuration written	2 owned types, 1 mutation entry, 1 aggregate, owner lookup	8 permission entries, 4 rule functions
One configuration entry removed		
Missing mutation entry	0 leaks (derived or denied)	that write executes (1 leak)
Missing type entry	type unprotected (9/13 or 1/13 leak)	0–1 leak (overlapping rules)
With deny fallback	—	0 leaks; 1–4 of 5 legitimate requests blocked

scenarios, judged by one oracle: an attack leaks if a non-owned object is returned or the victim object changes; a benign request fails if the caller's own objects are missing.

Both systems stop all 13 attacks when fully configured. graphql-shield's failure is availability: in `children: [ResourceType!]!`, denying the one foreign child cannot null that item, so the error nulls the whole folder and its owner loses their own two documents. Layer 2 removes the item before nullability is enforced. Under configuration mistakes neither approach dominates (Table V). GraphQL-Shield fails closed on mutations, but an owned type left out is a single point of failure that leaks up to 9 of 13 attacks; the startup lint reports such omissions. graphql-shield's overlapping rules tolerate one missing read rule, but a missing mutation rule executes the write under its default allow fallback, and its deny fallback blocks legitimate requests instead.

E. RQ5: Latency

All configurations of one request type are timed in the same run and interleaved: after 30 warm-up requests, each of 10 repeats times 100 requests of every configuration, in a fresh random order, after 10 untimed ones. Drift on the machine then affects every configuration alike, and the overhead is estimated from the per-repeat difference to the unprotected endpoint, with a 95% confidence interval (t, 9 degrees of freedom). An earlier, sequential version of this benchmark produced overheads that changed sign between runs.

With both layers, an owned lookup took 10.4 ms against 9.4 ms unprotected, a paired overhead of $+0.99$ ms (CI $[-0.36, 2.35]$); a relationship query $+0.86$ ms ($[0.32, 1.40]$); and an update $+0.00$ ms ($[-1.28, 1.29]$). Protection therefore costs about 1 ms per request on this machine, and at most 2.4 ms. Three fixes found by profiling brought it there. Pooling database connections cut both-layer latency 3.1 to 6.0 times (Fig. 2). The risk score ran an unindexed count over the whole audit log for every checked object, so latency grew with the log; an index and an existence probe made it constant. Finally, the advisory predictor cost about 4.4 ms of Layer 1's 5.3 ms, mostly starting XGBoost's thread pool for a one-row prediction; scoring on one thread and caching the probability per feature vector, together with caching parsed queries, removed nearly all of it.

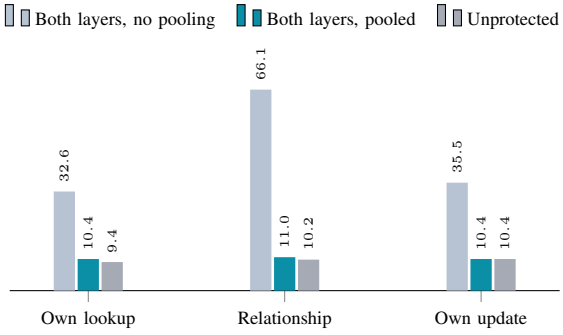


Fig. 2. Mean latency per request, in-process (10×100 interleaved requests each).

TABLE VI
LATENCY (MS) AS A REQUEST RETURNS MORE OBJECTS

Children	Unprotected	One by one	Batched
1	6.9	8.8	7.8
10	8.5	9.9	9.2
50	9.2	18.1	12.8
200	11.7	44.6	26.3

Scaling. A folder with N children, one in ten foreign, was queried with Layer 2 checking the list one item at a time or with one batched owner lookup and one audit transaction, interleaved with the unprotected endpoint (Table VI). Batching matters as lists grow: at 200 children it cut the protected request from 44.6 ms to 26.3 ms, against 11.7 ms unprotected. The remaining cost grows linearly with the number of returned objects, because each is checked and audited.

For reference, graphql-shield on Node.js added 15% to an owned lookup, 62% to a relationship query and 4% to an update over its own unprotected server (5×2,000 requests, means of 0.21–0.41 ms). The runtimes differ, so only the relative overheads are comparable, and only loosely.

F. RQ6: Self-repairing configuration

We planted 10 configuration mistakes a developer could make (Table VII): an identifier argument misnamed or missing, three mutation targets missing, and five owned types left out. For each, the same seeded traffic of 400 requests from four callers ran three times on fresh databases: with the correct configuration, with the mistake, and with the first verified repair applied. A run's outcome is the status and body of every response, and the configuration counts as restored when the repaired run's outcome equals the correct one's exactly. The correct configurations served as controls. To test verification itself, we also proposed 11 plausible wrong repairs per seed, such as reading `name` as an identifier or protecting a type with another type's owner table.

TABLE VII
SELF-REPAIR: 10 CONFIGURATION MISTAKES, 3 SEEDS OF 400 REQUESTS

Measure	Result
Mistakes detected	30 / 30
First verified repair is the correct one	30 / 30
Restored exactly, in one round	30 / 30
Responses changed by a mistake	23–207 per run
Repairs proposed for correct configurations	0 (6 runs)
Legitimate requests blocked by verified repairs	0
Wrong (decoy) repairs refused	33 / 33

Every mistake was detected, its first verified repair was the expected one, and applying it restored every response. In the clinic, removing the identifier argument also left `updateRecord` without a target, so a second verified repair registered it; the first already restored the configuration. The decoys were refused because their new identifiers were not recognised by the owner lookup, or because they blocked requests Layer 2 had let through, with one exception that changed the method. In the first decoy run, protecting `UserType` with the `Resource` owner table passed verification: the demo's user identifiers 101–104 coincide with resources owned by the same users, so replay could not tell the two lookups apart. We therefore added the rule that a repair may not reuse another owned type's ownership key, after which all decoys were refused. Replay can only judge repairs on the requests it has seen, and data where unrelated identifiers coincide is where it is weakest.

VIII. DISCUSSION AND LIMITATIONS

External validity. Both applications, the Graphene and Ariadne examples, the injected faults and all attack scenarios were written by the authors. The results show the mechanism works on these applications, not that it generalises. We did not evaluate on third-party applications or with independently written attacks, and we did not run black-box testers such as GraphQLer against the gateway; both are the most important future work.

Synthetic traffic. The predictor and the fault-injection study use generated requests from a small world. In this world a coverage gap follows almost entirely from query structure, so F1 near 1.0 reflects an easy target, not real-world accuracy. The fault study measures whether the evidence carries the signal, not how often real Layer 1 bugs occur.

Configuration and self-repair. An owned type left out is unprotected by both layers until the lint flags it and a repair is applied; a type with no lookup by identifier is found only through the lint's naming heuristic. Self-repair fixes configuration, not code, and its mistakes, traffic and decoys were written by the authors. The lint narrows this, but it combines naming with lookups by identifier and can still miss a type reached only through relationships. Only owner-or-administrator policies are supported; sharing, teams and attribute-based rules fit into Layer 2's per-object check but are not implemented.

Residual channels. The timing measurement is in-process; over a network an attacker averages more samples but also faces more noise, which we did not model. Moving a deleted folder's foreign children to the top level changes where they are filed, a choice the folder's owner makes for their own folder, but their owners may not expect it.

Measurement. Latency was measured in-process on a laptop; network latency, absent here, would make the roughly 1 ms overhead relatively smaller. graphql-shield was reimplemented on a copy of our schema in its documented style; other rule sets could perform differently, and its Node.js latency is not comparable in absolute terms with our Python numbers.

IX. CONCLUSION

Enforcing GraphQL object ownership at two independent points stops BOLA in every form we tested and lets a gateway measure its own coverage. A pre-execution layer alone misses objects reached through relationships; an execution-time layer catches them and records each object the first layer never saw. Planting 17 bugs in the first layer showed that this record detects every one, that a one-line rule over it predicts the affected requests, and that simple models name the broken construct as reliably as gradient boosting. The same record also repairs a mistaken configuration: every one of 10 planted mistakes was restored by its first verified repair, and verification refused every wrong one. Stating non-interference exposed and closed four leak channels in our own system, and the remaining responses show no timing difference. The layers are configured from the schema and a small ownership map, run on four Python GraphQL frameworks, and avoid the availability loss per-field rules suffer under null propagation. Evaluating on third-party applications with independently written attacks is the next step.

AVAILABILITY

Source code, the fault-injection harness and the results reported here are released under the MIT licence.

ACKNOWLEDGMENT

Parts of the implementation, evaluation scripts and text were drafted with the assistance of an AI coding assistant (Claude, Anthropic). The authors reviewed and are responsible for all content, and all reported numbers come from the released scripts.

REFERENCES

[1] OWASP Foundation, "OWASP API security top 10 – 2023: API1:2023 broken object level authorization," https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/, 2023.

[2] O. Hartig and J. Pérez, "Semantics and complexity of GraphQL," in *Proceedings of the 2018 World Wide Web Conference (WWW)*, 2018, pp. 1155–1164.

[3] "CVE-2022-39275: Saleor mutations did not validate object ID types," https://db.gcve.eu/vuln/CVE-2022-39275, 2022.

[4] GraphQL Foundation, "GraphQL specification: Handling field errors," https://spec.graphql.org/.

[5] O. Tsai, J. Li, T. T. Cheung, L. Huang, H. Zhu, J. Xiao, I. Sharafaldin, and M. A. Tayebi, "GraphQLer: Enhancing GraphQL security with context-aware API testing," arXiv:2504.13358, 2025.

[6] J. Barach, "AuthProbe: Specification-driven, multi-identity detection of broken object-level authorization in recruitment API," arXiv:2607.20574, 2026.

[7] A. Santos Filho, R. J. Rodríguez, and E. L. Feitosa, "Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored Petri nets transformation," *International Journal of Information Security*, vol. 24, no. 2, p. 83, 2025.

[8] A. Wu, Z. Feng, R. Feng, Z. Xing, and Y. Liu, "Rethinking broken object level authorization attacks under zero trust principle," *ACM Transactions on Software Engineering and Methodology*, 2026.

[9] L. Lambers, L. Sakizloglou, T. Khakharova, and F. Orejas, "Taint analysis for graph APIs focusing on broken access control," *Logical Methods in Computer Science*, vol. 22, no. 1, 2026.

[10] "graphql-shield: GraphQL server permissions as another layer of abstraction," npm package, version 7.6.5, https://www.npmjs.com/package/graphql-shield.

[11] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, "A principled approach to GraphQL query cost analysis," in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 257–268.

[12] E. Wittern, A. Cha, J. C. Davis, G. Baudart, and L. Mandel, "An empirical study of GraphQL schemas," in *Service-Oriented Computing (ICSOC)*, ser. Lecture Notes in Computer Science, 2019, pp. 3–19.

[13] J. P. Anderson, "Computer security technology planning study," U.S. Air Force Electronic Systems Division, Tech. Rep. ESD-TR-73-51, 1972.

[14] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[15] J. A. Goguen and J. Meseguer, "Security policies and security models," in *IEEE Symposium on Security and Privacy*, 1982, p. 11.

[16] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[17] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[18] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279.

[19] D. R. MacIver, Z. Hatfield-Dodds, and many contributors, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[20] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, "Snorkel: Rapid training data creation with weak supervision," *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.

[21] B. Kaur, "Broken object level authorization in the wild: An empirical taxonomy from 100+ bug bounty disclosures," arXiv:2605.25865, 2026.

[22] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[23] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.